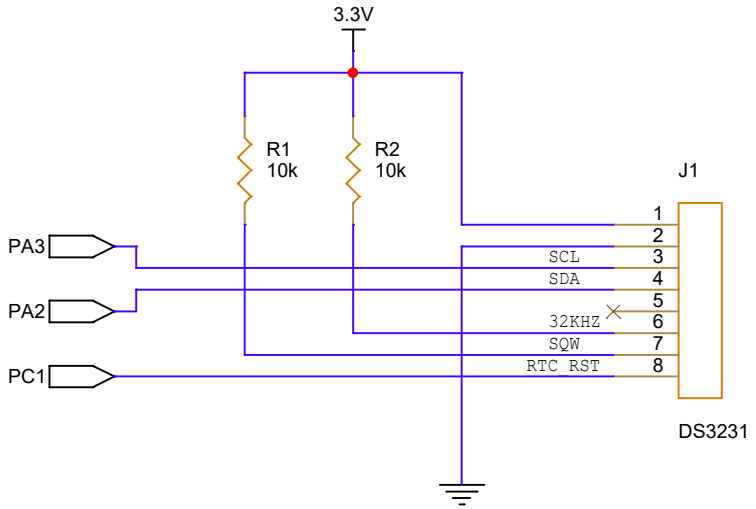
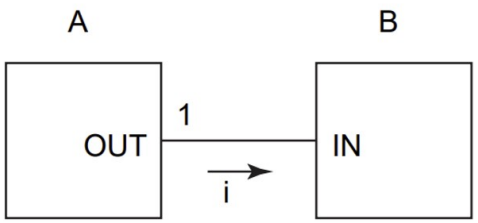


Title: Laboratory 10: Serial Alarm Real-Time Clock		
Size: A	Document Number	Rev: 0.1
Date: Saturday, April 13, 2024	Sheet 1 of 1	

Interface Checklist (revised 10/09/17)
ESE280 Ken Short

A = AVR128DB48

B = DS3231

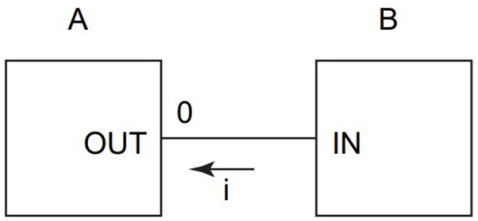


$V_{OHmin(A)} > V_{IHmin(B)}$

2.6V > 2.31V

$I_{OHmax(A)} > I_{IHmax(B)}$

6mA > 1μA

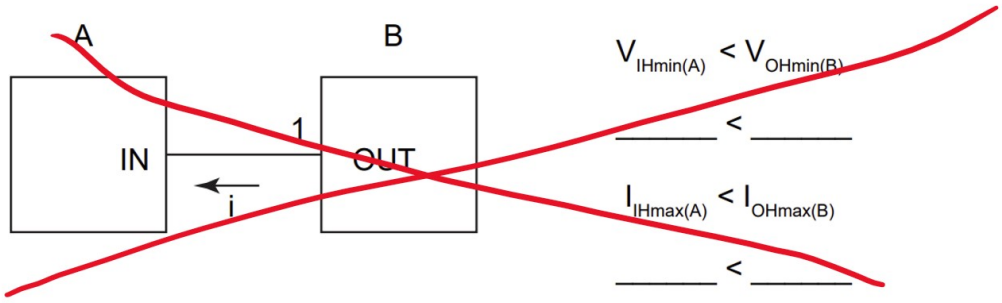


$V_{OLmax(A)} < V_{ILmax(B)}$

0.6V < 0.99V

$I_{OLmax(A)} > I_{ILmax(B)}$

10mA > 1μA



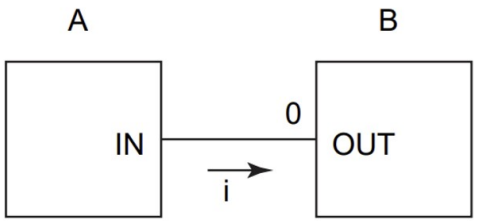
$V_{IHmin(A)} < V_{OHmin(B)}$

_____ < _____

$I_{IHmax(A)} < I_{OHmax(B)}$

_____ < _____

External Pull-up Resistor used



$V_{ILmax(A)} > V_{OLmax(B)}$

0.99V > 0.4V

$I_{ILmax(A)} < I_{OLmax(B)}$

1μA < 3mA

```
//*****************************************************************************
//
// File Name            : "DS3231_write_read_test.c"
// Title                : LAB10 - Task02
// Date                 : 4/13/2024 2:27 PM
// Version              : 1.0
// Target MCU           : AVR128DB48
// Target Hardware      : AVR128DB48 Curiosity Nano Board
// Author               :
// DESCRIPTION:
//       Program to write a value into one of the alarm registers and then
//       calls the function read_RTC to read that same register and verify
//       the I2C communications between the AVR128DB48 and the DS3231.
//
// Warnings : none
// Restrictions : none
// Algorithms : none
// References :
//
// Revision History : Initial version
//
//
//*****************************************************************************

#define F_CPU       4000000     // 4MHz default clock
#define CLK_PER     4000000     // 4MHz default clock

/* Calculations for I2C baud prescalar value*/
#define TWI0_BAUD(F_SCL, T_RISE)    ((((((float)CLK_PER / (float)F_SCL)) - 10 -
  ((float)CLK_PER * T_RISE))) / 2)

#include <avr/io.h>
#include <stdio.h>
#include <util/delay.h>
#include <string.h>

#define DS3231_ADDR 0x68        // DS3231 address
#define DS3231_BAUD 400000      // 400KHz I2C clock, maximum frequency for DS3231
#define TEMPERATURE_ADDR 0x00   // slave address of temperature register

/* Function Prototypes*/
void I2C_rtc_DS3231_config(void);
void write_RTC (uint8_t slave, uint8_t reg_RTC, uint8_t data_RTC);
uint8_t read_RTC (uint8_t slave, uint8_t reg_RTC);

int main(void)
{
    volatile DS3231_RTC_value;
    I2C_rtc_DS3231_config();
```

```
    while (1)
    {
        write_RTC(DS3231_ADDR, 0x00, 0x00);
        _delay_ms(1);
        DS3231_RTC_value = read_RTC(DS3231_ADDR, 0x00);
    }

    return 0;
}

//***************************************************************************
// Function : void I2C_rtc_DS3231_config(void)
// Date and version : 041024, version 1.0
// Target MCU : AVR128 @ 4MHz
// Author : Ken Short
// DESCRIPTION
// This function configures an AVR128DB48 operated at
// 4 MHz to communicate with the DS3231
// SCL must be operated at the maximum possible frequency for
// the DS3231.
//***************************************************************************

void I2C_rtc_DS3231_config(void)
{
    TWI0.MBAUD = (uint8_t)TWI0_BAUD(DS3231_BAUD, 0.282); // 400KHz I2C clock,
      maximum frequency for DS3231
    TWI0.MCTRLA = TWI_ENABLE_bm; // Enable TWI master bit0
    TWI0.DBGCTRL = TWI_DBGRUN_bm; // Debug run
    TWI0.MSTATUS = TWI_BUSSTATE_IDLE_gc; // Force bus state to idle
}

//***************************************************************************
// Function: void write_RTC (uint8_t slave, unit8_t reg_RTC, uint8_t data_RTC)
//
// Target MCU : AVR128DB48 @ 4MHz
// Target Hardware ;
// Author : Ken Short
// DESCRIPTION
// This function writes data to a register in the RTC. To accomplish this, it
// must first write the DS3231's slave address, then the register's pointer
// address, and finally the data.
//***************************************************************************

void write_RTC (uint8_t slave, uint8_t reg_RTC, uint8_t data_RTC)
{
    // wait until idle
    while(!(TWI0.MSTATUS & TWI_BUSSTATE_IDLE_gc));
```

```
    TWI0.MADDR = slave << 1; // send slave address for write
    while(!(TWI0.MSTATUS & TWI_WIF_bm)); // WIF flag, wait until slave address sent

    // The next write clears the WIF flag
    TWI0.MDATA = reg_RTC; // send register pointer address
    while(!(TWI0.MSTATUS & TWI_WIF_bm)); // WIF flag, wait until reg_RTC sent

    // The next write clears the WIF flag
    TWI0.MDATA = data_RTC; // send data
    while(!(TWI0.MSTATUS & TWI_WIF_bm)); // WIF flag, wait until data sent

    // The next write clears the WIF flag
    TWI0.MCTRLB |= TWI_MCMD_STOP_gc; // issue a stop

    return 0;
}

//***************************************************************************
// Function: uint8_t read_RTC (uint8_t slave, uint8_t reg_RTC)
// Target MCU : AVR128DB48 @ 4MHz
// Author : Ken Short
// DESCRIPTION
// This function reads data from a register in the RTC. To accomplish this, it
// must first write the DS3231's slave address, then its pointer address, and
// finally read the data.
//***************************************************************************

uint8_t read_RTC (uint8_t slave, uint8_t reg_RTC)
{
    // wait until idle
    while(!(TWI0.MSTATUS & TWI_BUSSTATE_IDLE_gc));

    TWI0.MADDR = ((slave << 1) | 0x01); // send slave address and read command
    while(!(TWI0.MSTATUS & TWI_RIF_bm)); // RIF flag, wait until byte is received,
      WIF flag does not work here

    // The next write clears the WIF flag
    TWI0.MDATA = reg_RTC; // send register pointer address
    while(!(TWI0.MSTATUS & TWI_WIF_bm)); // WIF flag, wait until reg_RTC sent

    uint8_t temp_RTC_value = TWI0.MDATA; // clears the RIF flag
    TWI0.MCTRLB = TWI_MCMD_RECVTRANS_gc; // MCMD - issue ack followed by a byte
      read operation

    TWI0.MCTRLB = 0x07; // MCMD issue nack followed by a stop

    // read data from received data buffer
    return (uint8_t)(temp_RTC_value);
}
```